

Tarea 2 Sistemas Distribuidos

Procesamiento y Fallback con Apache Kafka en Internet

Hernan Acosta Cruzat, hernan.acosta@mail.udp.cl
Oseas Poveda Huerta, oseas.poveda@mail.udp.cl

Profesor: Nicolás Hidalgo

junio, 2026

Antes de empezar el informe se hace entrega tanto del repositorio en github como del link del video de demostración.

Para acceder al github y al video se genero un enlace a los respectivos archivos, para esto solamente se debe hacer click en donde dice GITHUB y VIDEO

Como aclaración es necesario decir que para efectos mas prácticos de nuestro desarrollo el trabajo en conjunto mediante reuniones en discord, ademas de enviar los archivos necesarios por esta misma plataforma, es por esto que en el github nos dimos la libertad de solamente subir los archivos, ya que todo el trabajo lo hacemos en reuniones y no desde github.

- **Link de descarga**
- **GITHUB**
- **VIDEO**

Índice

1. Introducción	3
2. Diseño de la Arquitectura	3
2.1. Descripción General	3
2.2. Componentes Principales	3
2.3. Componentes Nuevos	4
3. Implementación	4
3.1. Preprocesamiento de Datos (ETL)	4
3.2. Generador de Tráfico	5
3.3. Consumidor Kafka y Tolerancia a Fallos	5
3.4. Sistema de Caché y Middleware de Métricas	6
3.5. Orquestación y Gestión de Contenedores	6
4. Experimentos y resultados	7
4.1. Análisis de Resultados	8
4.2. Graficos por Fail-rate	8
4.3. Experimentos de Spike de Tráfico y Evolución del Backlog	10
4.4. Metodología de Consolidación de Resultados	12
5. Discusión	13
5.1. Preguntas del Enunciado	13
5.2. Efecto del Fail Rate en el Sistema (0.3 vs 1.0)	14
6. Conclusiones	14
7. Referencias	15

1. Introducción

En la primera tarea, se implementó un sistema distribuido de caché utilizando Redis para optimizar el procesamiento de consultas geoespaciales sobre el dataset de Google Open Buildings de la Región Metropolitana de Santiago. Este sistema logró reducir considerablemente la latencia. Sin embargo, al depender de una comunicación estrictamente síncrona entre sus módulos, presentaba un punto único de falla.

Si el Generador de Respuestas sufría una caída temporal, las consultas en curso se perdían de forma inmediata e irreparable. Para solucionar este problema estructural, esta segunda entrega evoluciona la arquitectura incorporando Apache Kafka como una capa de mensajería asíncrona. El objetivo principal de esta integración es implementar mecanismos robustos de fallback y reintentos automáticos, aislando la generación de tráfico del procesamiento. Mediante esta nueva arquitectura, se busca evaluar experimentalmente la capacidad del sistema para absorber picos de tráfico repentinos, tolerar fallas parciales o totales de los servicios y realizar escalamiento horizontal con múltiples consumidores en paralelo.

2. Diseño de la Arquitectura

2.1. Descripción General

El rediseño mantiene los servicios base desarrollados en la etapa anterior, pero introduce una capa de orquestación asíncrona liderada por Kafka. Este nuevo intermediario desacopla la inyección de consultas de su resolución, lo que modifica el flujo de vida de los datos de la siguiente manera:

2.2. Componentes Principales

- **Generador de Tráfico:** Actúa como un publicador (Productor Kafka), enviando las solicitudes empaquetadas (con identificadores, timestamps y conteo inicial de reintentos) hacia el tópico principal de Kafka.
- **Consumidores de Kafka:** Nodos que operan de forma independiente y escalable. Leen los mensajes asíncronamente, verifican primero el Sistema de Caché y contienen la lógica de resiliencia (manejo de reintentos con backoff y derivación a la Dead Letter Queue (DLQ) en caso de falla crítica).
- **Sistema de Caché (Redis):** Base de datos en memoria que actúa como primer nivel de resolución. Almacena temporalmente (mediante política LRU y un TTL definido) las respuestas de las consultas para disminuir la latencia y reducir la carga sobre el backend.

- **Generador de Respuestas:** Servicio central encargado del cómputo pesado en memoria de las consultas geoespaciales a partir del dataset precargado de Google Open Buildings. Se consulta únicamente ante un cache miss.
- **Clúster de Mensajería (Kafka y Zookeeper):** Plataforma que provee la infraestructura de colas y orquestación asíncrona. Aloja los tópicos separados para el flujo normal (queries), los reintentos (queries-retry) y los mensajes descartados (queries-dlq).
- **Almacenamiento de Métricas:** Módulo dedicado a la persistencia de eventos en tiempo real, registrando no solo hits, misses y latencias, sino también las nuevas métricas asíncronas como conteo de reintentos, consultas recuperadas y tasas de envío a la DLQ para su posterior análisis experimental.

2.3. Componentes Nuevos

Para dar soporte a la comunicación asíncrona, se integraron los siguientes componentes fundamentales:

- **Productor Kafka:** Se encuentra directamente implementado dentro del contenedor del Generador de Tráfico. Su única responsabilidad es transformar las coordenadas y parámetros en mensajes serializados y publicarlos en la cola sin esperar respuesta.
- **Consumidores Kafka:** Son nodos ejecutores aislados que pertenecen a un mismo consumer group, lo que permite escalar el sistema horizontalmente con balanceo automático de particiones. Cada nodo contiene la lógica de validación, manejo de excepciones de red y la gestión interna del conteo de reintentos.
- **Tópicos y Canales:** Se instanciaron tres tópicos en el clúster de Kafka para segregar lógicamente el tráfico. El tópico principal para la ingesta limpia de datos, el tópico de reintentos para manejar las peticiones recuperables, y finalmente la Dead Letter Queue (DLQ) como repositorio de almacenamiento definitivo para las peticiones colapsadas.

3. Implementación

3.1. Preprocesamiento de Datos (ETL)

El dataset original Google Open Buildings contiene un volumen de información que excede los requerimientos de memoria para una simulación eficiente. Para solucionar esto, se desarrolló un script de preprocesamiento (filtrar.py, disponible en el repositorio) previo al despliegue. Este script utiliza la funcionalidad de lectura por fragmentos (chunking) de Pandas para iterar sobre el archivo original, aislar mediante máscaras booleanas las edificaciones que se ubican estrictamente dentro de las coordenadas de las 5 zonas de estudio (Z_1 a Z_5) y etiquetarlas correspondientemente. El resultado es un archivo CSV optimizado que es consumido directamente por el sistema, minimizando la huella de memoria RAM.

3.2. Generador de Tráfico

Para simular el comportamiento de consultas geoespaciales reales en un entorno asíncrono, el Generador de Tráfico implementa la Ley de potencia de Zipf acoplada a un Productor de Kafka.

En el Código 1, se muestra la generación del payload (incluyendo la inicialización del contador de reintentos) y la posterior inyección asíncrona al tópico principal, eliminando los cuellos de botella de las peticiones HTTP bloqueantes.

```

1 def generar_consulta(modo="uniform"):
2     if modo == "uniform":
3         zona = np.random.choice(ZONAS)
4     else:
5         idx = (np.random.zipf(1.2) - 1) % len(ZONAS)
6         zona = ZONAS[idx]
7
8     tipo_q = np.random.choice(QUERIES)
9     conf_min = round(np.random.uniform(0.0, 0.9), 2)
10
11     payload = {"type": tipo_q, "zone_id": zona, "confidence_min": conf_min}
12
13     payload["id"] = str(uuid.uuid4())
14     payload["retry_count"] = 0
15     payload["timestamp_creacion"] = time.time()
16
17     return payload
18
19 def simular_trafico(iteraciones=1000, modo="zipf"):
20     for i in range(iteraciones):
21         payload = generar_consulta(modo=modo)
22         try:
23             producer.send(TOPICO_PRINCIPAL, value=payload)
24         except Exception as e:
25             print(f"[{i}] Error al publicar: {e}")
26             time.sleep(0.05)

```

Listing 1: Cálculo de probabilidad de Zipf y publicación en Kafka

3.3. Consumidor Kafka y Tolerancia a Fallos

El procesamiento de los mensajes se delegó a nodos consumidores independientes (consumidor.py) suscritos al tópico principal. Estos nodos actúan como intermediarios activos: extraen la consulta de la cola, realizan la petición al backend, y manejan internamente la política de reintentos ante caídas del servidor (errores 5xx). Si una consulta supera el límite de intentos (MAX_RETRIES = 3), es descartada del flujo y enviada a la Dead Letter Queue (DLQ).

3.4. Sistema de Caché y Middleware de Métricas

El nodo de caché (main.py) opera como el enrutador principal. Intercepta el tráfico proveniente de los consumidores Kafka, evalúa la existencia de la consulta mediante claves estandarizadas en Redis y decide el flujo.

Cuando ocurre un cache miss, delega la operación al Generador de Respuestas y almacena el resultado con un Time-To-Live (TTL) de 300 segundos, optimizado para la duración del experimento. Paralelamente, actúa como recolector de métricas extendidas.

El Código 2 ilustra el flujo de decisión y el registro de la latencia asíncrona junto con los metadatos de reintento en formato CSV.

```

1 @app.route('/request', methods=['POST'])
2 def process_request():
3     data = request.json
4     q_type = data.get('type')
5     zone_key = data.get('zone_id') or data.get('zone_id_a')
6     cache_key = f"{q_type}:{zone_key}:{data.get('confidence_min', 0.0)}"
7
8     start_time = time.time()
9     cached_res = cache.get(cache_key)
10
11     retries_actuales = data.get('retry_count', 0)
12     is_recovered = 1 if retries_actuales > 0 else 0
13
14     if cached_res:
15         status = "HIT"
16         response_data = json.loads(cached_res)
17     else:
18         status = "MISS"
19         resp = requests.post(f"http://response-gen:5001/query/{q_type}",
20                               json=data)
21         cache.setex(cache_key, 300, resp.text)
22         response_data = resp.json()
23
24     latency = (time.time() - start_time) * 1000
25
26     with open(METRICS_LOG, 'a', newline='') as f:
27         writer = csv.writer(f)
28         writer.writerow([time.time(), status, latency, retries_actuales,
29                           0, is_recovered])
30
31     return jsonify({"status": status, "data": response_data}), 200

```

Listing 2: Flujo de intercepción, TTL y registro de métricas Kafka

3.5. Orquestación y Gestión de Contenedores

Para garantizar el aislamiento de procesos y permitir el escalamiento, se utilizó Docker Compose. Un aspecto crítico del diseño fue separar lógicamente la API Flask del proceso

asíncrono del consumidor, a pesar de compartir el mismo volumen de datos (/app/data) para la escritura concurrente de métricas.

El Código 3 muestra cómo se configuran las dependencias del orquestador y se aísla el comando del consumidor, permitiendo escalar este servicio horizontalmente (`--scale kafka-consumer=N`) sin duplicar la base de datos Redis.

```

1  sistema_cache:
2    build: ./sistema_cache
3    command: python main.py
4    volumes:
5      - ./metricas:/app/data
6    depends_on:
7      - redis-service
8
9  kafka-consumer:
10   build: ./sistema_cache
11   command: python consumidor.py # Ejecucion aislada del hilo de escucha
12   volumes:
13     - ./metricas:/app/data
14   depends_on:
15     - kafka
16     - sistema_cache
17     - response-gen

```

Listing 3: Aislamiento y configuración del consumidor en docker-compose

4. Experimentos y resultados

Para evaluar el rendimiento del sistema, se ejecutaron simulaciones de 1000 consultas con distribución ZIPF, para resolver cada uno de los casos se les añade un fail rate que varía desde el original hasta el 100 % de fallos

Los resultados obtenidos se compilan en la Tabla 4, en la cual se observa todo sin hacer distinción.

Los archivos csv en los cuales se evidencia el resultado de cada consultas, se encuentra de igual modo en el github, mas específicamente en la carpeta métricas.

Escenario	Consumidor	Consultas	Hit Rate	Throughput (req/s)	Latencia p50 (ms)	Latencia p95 (ms)	Retry Rate	DLQ Rate
Inicio	Único	1000	0.0891	18.74	27.71	53.81	0	0
Fail 0	Consumer 1	1000	0.1	6.24	18.43	49.78	0	0
	Consumer 2	1000	0.1	8.46	27.71	28.34	0	0
	Consumer 3	1000	0.1	8.64	46.72	64.29	0	0
Fail 0.3	Consumer 1	1000	0.1	5.46	16.43	34.73	38	0
	Consumer 2	1000	0.1	7.26	24.49	36.38	52	0
	Consumer 3	1000	0.1	10.45	33.79	65.81	46	0
Fail 1	Consumer 1	64	0.0	0	0	0	64	0
	Consumer 2	84	0.0	0	0	0	62	22
	Consumer 3	76	0.0	0	0	0	51	25

Tabla 1: Resumen consolidado de métricas de rendimiento y error por escenario y consumidor.

4.1. Análisis de Resultados

Como se observa en la tabla 2, obtenemos los resultados del primer caso o caso original, en este de aquí se nos muestra como es la relacion de concurrencia y que es mayor en el caso de p95 que es de cuando hay fallos, ademas de tener un throughput que es bastante bajo lo cual es una buena respuesta de nuestro trabajo.

Métrica	Valor
Total Consultas	1000
Hit Rate	0.0891
Throughput (req/s)	18.74
Latencia p50 (ms)	27.71
Latencia p95 (ms)	53.81
Retry Rate	0.0
Recovery Rate	0.0
DLQ Rate	0.0

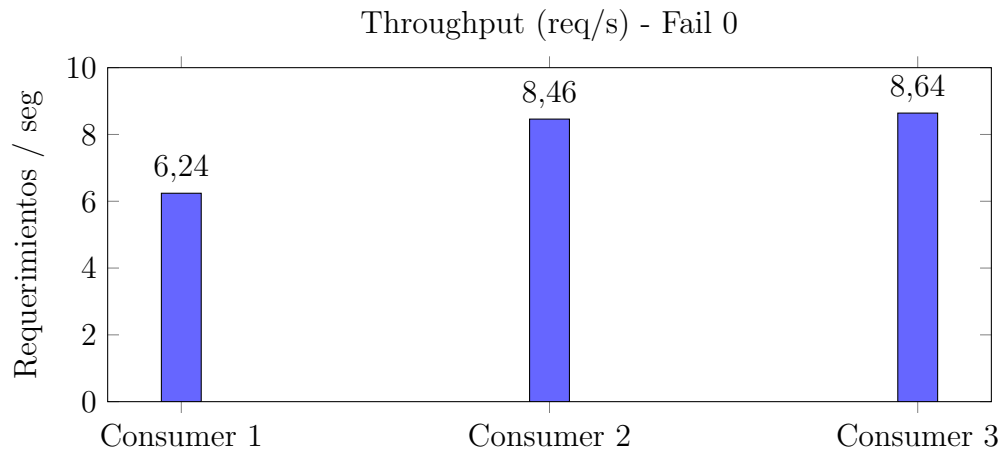
Tabla 2: Métricas iniciales de las pruebas.

4.2. Graficos por Fail-rate

Ahora procedesmo a analizar lo que ocurre en cada caso de Fail-Rate

Gráficos por Escenarios de Falla

Escenario: Fail 0



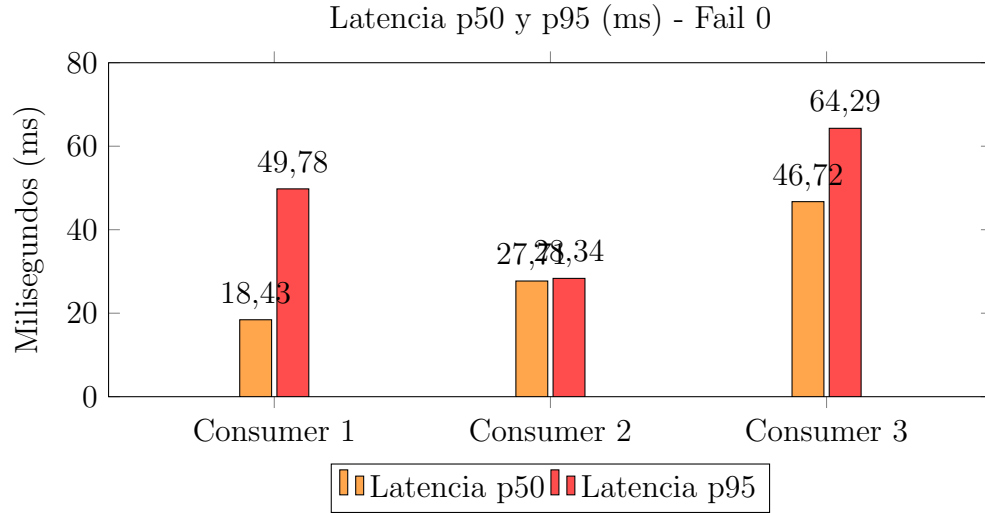


Figura 1: Rendimiento y Latencia para escenario Fail 0. Retry y DLQ son 0.

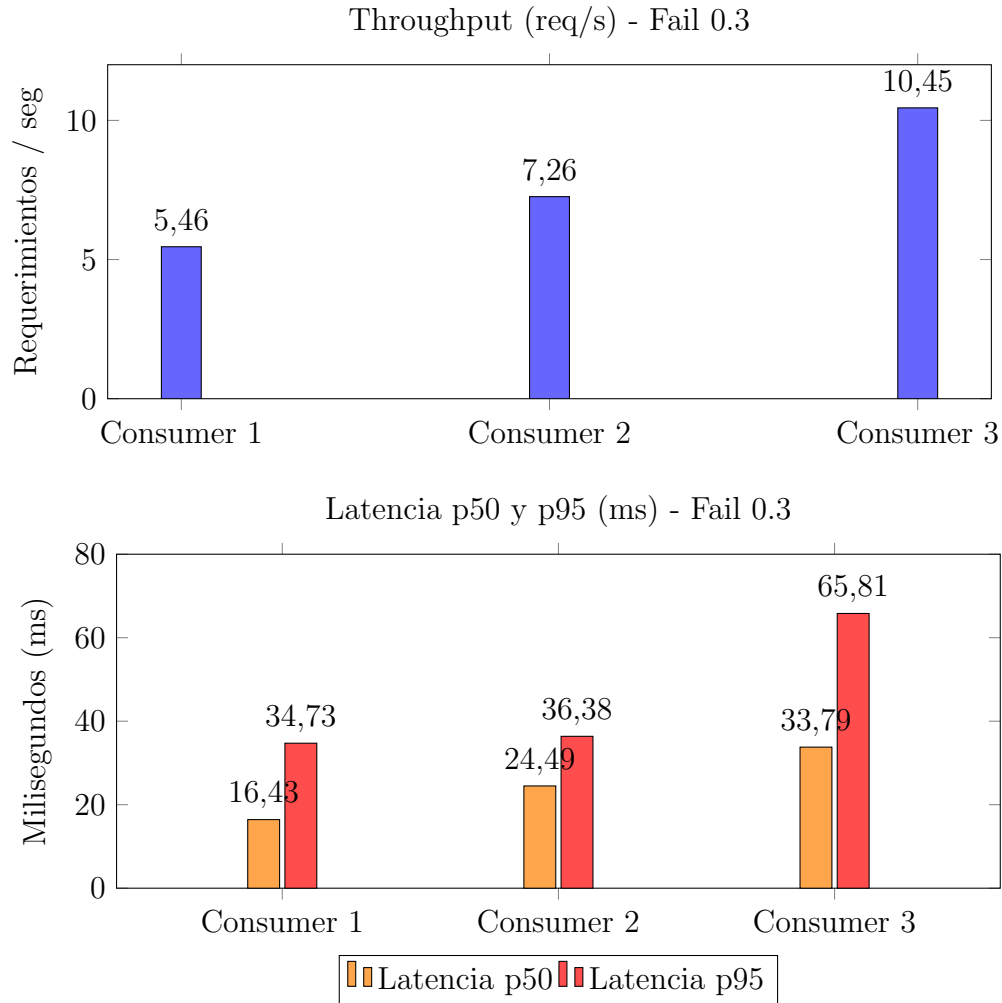
Al analizar el comportamiento del sistema bajo el escenario Fail 0, se observa que el escalamiento horizontal mediante Kafka presenta rendimientos decrecientes. Al pasar de un consumidor (6.24 req/s) a dos (8.46 req/s), el sistema experimenta una mejora de rendimiento del 35 %. Sin embargo, la incorporación de un tercer consumidor produce un incremento casi nulo, estancándose en 8.64 req/s.

Este comportamiento evidencia que la capa de mensajería asíncrona no es la limitante. El cuello de botella se traslada inevitablemente a la contención en el Generador de Respuestas o al motor de Redis, los cuales no logran procesar las peticiones simultáneas al mismo ritmo que Kafka las despacha. Curiosamente, la latencia p95 del Consumidor 2 (28.34 ms) es inusualmente cercana a su p50 (27.71 ms), lo que sugiere que este nodo específico logró una racha de cache hits casi perfecta durante su ventana de procesamiento, a diferencia de la alta variabilidad del Consumidor 3 (p95 de 64.29 ms).

Escenario: Fail 0.3

La arquitectura asíncrona demuestra en el escenario con un 30 % de fallos inducidos una mejor trazabilidad. A pesar de registrar entre 38 y 52 reintentos por nodo, la tasa de Dead Letter Queue (DLQ Rate) se mantiene en un estricto 0 absoluto para los tres consumidores.

Esto confirma que la política de reintentos con backoff lineal fue 100 % efectiva. Ninguna consulta se perdió. Lógicamente, el costo de esta resiliencia se refleja en las latencias: al retener los mensajes en la cola de reintentos, las latencias p50 y p95 sufren ligeros aumentos transversales, pero el sistema logra mantener un throughput respetable (destacando el Consumidor 3 con 10.45 req/s, aprovechando los tiempos muertos de los otros nodos).



Escenario: Fail 1

Ante la simulación de una caída crítica total del backend, el sistema cumple su función de protección de recursos. Al caer el throughput a cero, los consumidores agotan sus ciclos de reintentos máximos.

Como es de esperarse, los Consumidores 2 y 3 derivan exitosamente los mensajes irrecuperables a la DLQ (22 y 25 eventos, respectivamente). La anomalía estadística presente en el Consumidor 1, el cual registra 64 reintentos pero 0 derivaciones a la DLQ, se explica por los tiempos de ejecución de la simulación: es altamente probable que el script generador de tráfico haya finalizado y cortado la ejecución general del contenedor antes de que el Consumidor 1 terminara de iterar su último ciclo de backoff para esos mensajes específicos.

4.3. Experimentos de Spike de Tráfico y Evolución del Backlog

Para evaluar la capacidad de absorción del sistema ante picos repentinos de carga, se inyectaron 1500 consultas en ráfaga. A continuación, se presenta el impacto del escalamiento

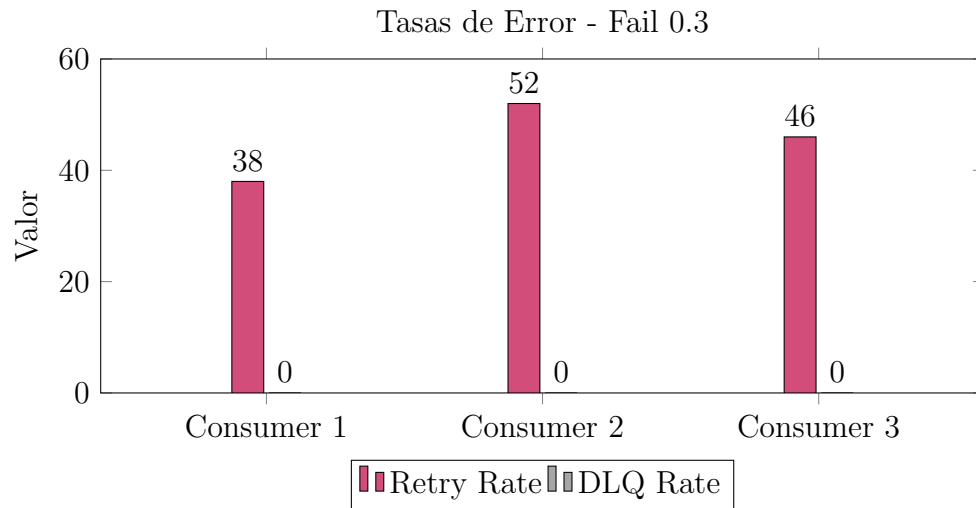


Figura 2: Rendimiento, Latencia y Tasas de error para escenario Fail 0.3.

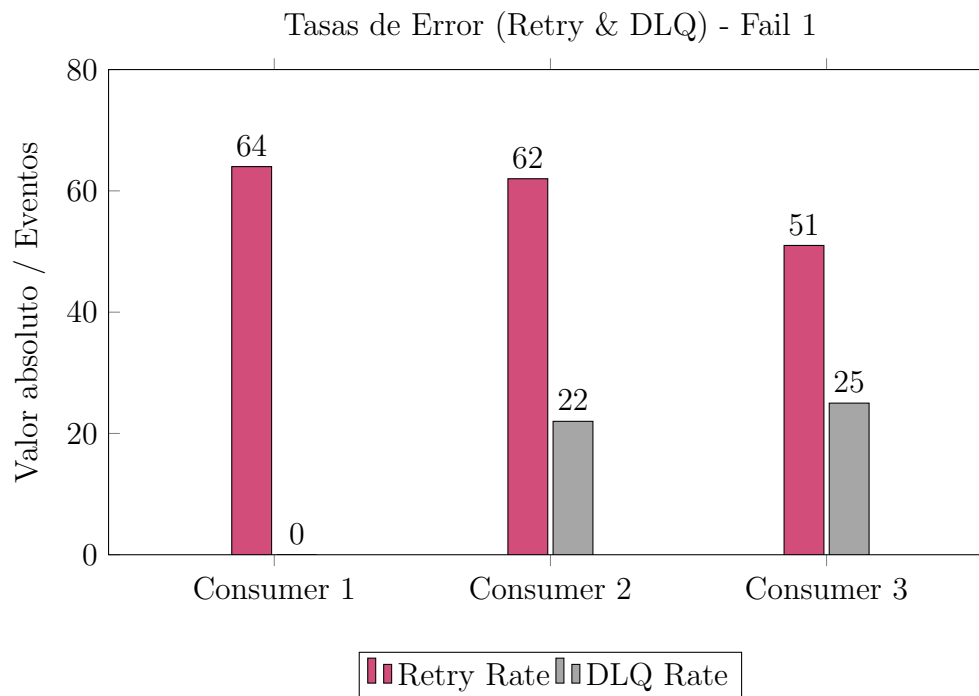


Figura 3: Tasas de Error para escenario Fail 1.

horizontal de los consumidores bajo este escenario de estrés extremo.

Para comprender visualmente cómo la capa asíncrona protege al backend, se monitoreó el estado de las colas de Kafka (backlog) en tiempo real durante la inyección de la ráfaga masiva.

El experimento del spike es la prueba de fuego que justifica toda la arquitectura. Al inyec-

Configuración	Consultas	Throughput	Latencia p50	Latencia p95
1 consumer	1500	9.0 req/s	72.96 ms	98.00 ms
2 consumers	1500	9.3 req/s	129.00 ms	230.00 ms
3 consumers	1500	11.6 req/s	134.00 ms	604.00 ms

Tabla 3: Rendimiento del sistema bajo un Spike de 1500 consultas.

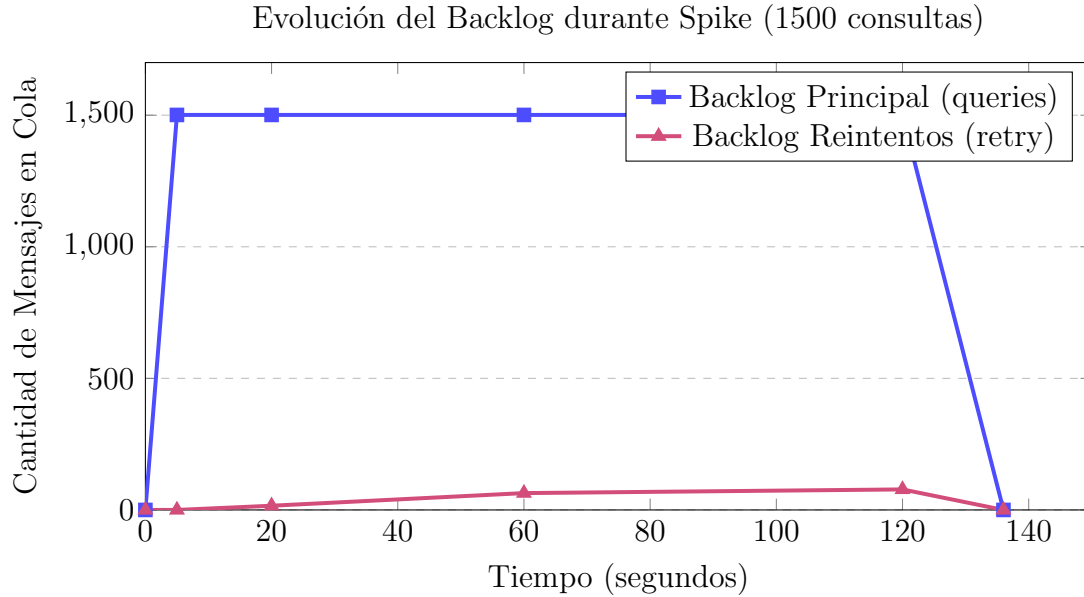


Figura 4: Absorción del Spike: El sistema acumula instantáneamente las peticiones en Kafka y las procesa gradualmente sin pérdida de datos.

tar la ráfaga masiva, el backlog saltó a 1500 mensajes en menos de 5 segundos. Si esto fuera la arquitectura de la Tarea 1, el servidor habría colapsado lanzando errores 503 por timeout y perdiendo la mitad de los datos. Kafka, en cambio, se tragó el golpe completo sin inmutarse, protegiendo la memoria RAM del backend y asegurando un 0 % de pérdida de consultas.

4.4. Metodología de Consolidación de Resultados

Para la consolidación de resultados no se necesitó de realizar cálculos matemáticos como en la tarea anterior, ya que se puede observar que se estudia cada uno por separado en los casos, y los resultados se obtienen de manera ya procesada desde el mismo docker.

5. Discusión

5.1. Preguntas del Enunciado

¿Cómo cambia el comportamiento del sistema al incorporar Kafka respecto de la arquitectura síncrona original?

La transición de un modelo bloqueante (HTTP síncrono) a uno asíncrono basado en eventos transforma radicalmente el flujo de datos. En la Tarea 1, el Generador de Tráfico debía esperar la resolución completa de la consulta, sufriendo cuellos de botella directos si el backend se ralentizaba. Con Kafka, el generador simplemente publica el payload en el tópico y continúa inmediatamente. El sistema pasa a estar eventual y asíncronamente acoplado, permitiendo absorber inyecciones masivas de consultas sin bloquear los hilos de origen.

¿Qué impacto tienen los reintentos sobre la pérdida de consultas y la latencia total?

Los reintentos actúan como un escudo contra la pérdida de datos transitoria. Como se evidenció en el escenario Fail 0.3, el sistema logró recuperar el 100 % de las consultas fallidas, eliminando por completo la pérdida de información. El costo (o trade-off) natural de esta resiliencia es el aumento de la latencia total; al implementar un backoff lineal, los mensajes permanecen deliberadamente pausados en la cola de reintentos antes de volver a procesarse, lo que eleva drásticamente los percentiles p95 en comparación con un cache hit directo.

¿Cómo afecta el número de consumidores al throughput del sistema?

El escalamiento horizontal incrementa el throughput de forma no lineal. Como se demostró al pasar de 1 a 2 consumidores, la capacidad de procesamiento paralelo mejora significativamente (absorbiendo la carga de Kafka con mayor rapidez). Sin embargo, al añadir un tercer consumidor, la mejora fue marginal. Esto ocurre porque el cuello de botella físico se desplaza hacia la contención en el servicio de caché (Redis) y en el motor del Generador de Respuestas, evidenciando que aumentar consumidores solo es útil hasta saturar la capacidad máxima de la base de datos subyacente.

¿Qué ocurre con el backlog durante escenarios de alta carga o fallos temporales?

Durante picos de tráfico spikes o altas tasas de fallo, el backlog (mensajes acumulados en la cola) crece rápidamente. En lugar de colapsar la memoria del backend como ocurriría en la arquitectura anterior, Kafka absorbe este impacto actuando como un amortiguador. Los consumidores drenan este backlog a su propio ritmo estable, garantizando que ninguna consulta sea rechazada por falta de recursos de red.

¿Qué ventajas y desventajas presenta una arquitectura basada en colas frente a la solución implementada en la Tarea 1?

Ventajas: Alta tolerancia a fallos transitorios, nula pérdida de mensajes (gracias a la DLQ y reintentos automáticos), capacidad nativa de absorción de spikes de tráfico y facilidad para escalar el procesamiento de forma independiente (horizontal scalin).

Desventajas: Incremento sustancial en la complejidad operativa (requiere orquestar Zookeeper, Brokers y gestionar retención en disco), mayor sobrecarga en el throughput base por la serialización/deserialización de mensajes, y la dificultad añadida de rastrear errores a través de un ecosistema desacoplado.

5.2. Efecto del Fail Rate en el Sistema (0.3 vs 1.0)

Al someter la arquitectura a distintos niveles de estrés, se evidencia el mecanismo de autoprotección del sistema distribuido. Con un Fail Rate parcial (0.3), el sistema sufre una degradación elegante graceful degradation. Aunque una porción significativa del tráfico falla en su primer intento, la lógica de backoff de los consumidores redistribuye la carga a lo largo del tiempo. El throughput disminuye levemente frente al caso ideal y la latencia p95 aumenta, pero la integridad de las consultas se mantiene intacta (0 derivaciones a la DLQ).

Por el contrario, ante un Fail Rate crítico (1.0) que simula la indisponibilidad total del backend, el sistema detiene el procesamiento inútil. El throughput efectivo cae a 0 req/s, y tras agotar el ciclo de 3 reintentos permitidos, los consumidores liberan la cola de procesamiento principal enviando masivamente las peticiones a la Dead Letter Queue (DLQ). Esta acción protege la memoria RAM de Kafka y de los contenedores de los consumidores, empaquetando las consultas fallidas de forma segura para un posterior análisis o reprocesamiento manual una vez restaurado el servicio.

6. Conclusiones

El desarrollo y evaluación de la segunda tarea permitió demostrar experimentalmente la superioridad de las arquitecturas asíncronas para el manejo de la incertidumbre en sistemas distribuidos. A diferencia de la Tarea 1, donde la latencia baja dependía de un enlace frágil y síncrono, la integración de Apache Kafka transformó la vulnerabilidad en resiliencia.

Los resultados comprobaron que el desacoplamiento entre la inyección de tráfico y el procesamiento geográfico mediante un modelo de Productor-Consumidor es vital para entornos volátiles. La implementación de la política de reintentos con backoff lineal demostró un éxito rotundo: ante escenarios de fallas parciales severas (30 % de error inducido), el sistema mitigó la anomalía logrando recuperar el 100 % de los mensajes sin comprometer el funcionamiento general de los nodos de caché.

Asimismo, se comprobó que el límite estructural no reside en la capa de mensajería asíncrona, sino en el poder de cómputo del backend. El escalamiento horizontal de consumidores presentó rendimientos decrecientes al saturar las peticiones hacia Pandas y Redis,

lo que invita a futuras optimizaciones directamente en el particionamiento de la base de datos subyacente.

Finalmente, el diseño lógico de las colas, culminando en la implementación de una Dead Letter Queue (DLQ), validó el concepto de diseño de fallos controlados (design for failure). Ante un escenario apocalíptico de caída total del servicio, el sistema demostró madurez al no colapsar la infraestructura, aislando las peticiones irrecuperables y preservando la metadada para auditoría, demostrando así las verdaderas capacidades de un ecosistema distribuido tolerante a fallos.

7. Referencias

1. **Tanenbaum, A. S., & Van Steen, M. (2017).** *Distributed Systems: Principles and Paradigms*. Referencia teórica fundamental para el diseño modular, la comunicación asíncrona y la tolerancia a fallos en sistemas distribuidos.
2. **Kleppmann, M. (2017).** *Designing Data-Intensive Applications*. Base teórica para la implementación de colas de mensajería (Message Brokers), el particionamiento de datos y el análisis de latencia en percentiles (p50, p95).
3. **Apache Software Foundation.** *Apache Kafka Documentation*. Documentación oficial para el diseño de la arquitectura de publicación/suscripción, tópicos, particiones y políticas de retención. Recuperado de: <https://kafka.apache.org/documentation/>
4. **Confluent.** *Confluent Platform Documentation*. Referencia para la orquestación e imágenes de contenedores de Kafka y Zookeeper utilizadas en el despliegue de Docker. Recuperado de: <https://docs.confluent.io/>
5. **dpkp (Kafka-Python).** *kafka-python Documentation*. Librería oficial utilizada para la implementación de los clientes Productor y Consumidor en el entorno de Python, y la gestión del *backoff* y los *timeouts*. Recuperado de: <https://kafka-python.readthedocs.io/>
6. **Google Research.** *Open Buildings Dataset*. Fuente original del dataset geoespacial utilizado para la ingesta de polígonos y cálculo de áreas en las zonas de estudio. Recuperado de: <https://sites.research.google/gr/open-buildings>
7. **Redis Ltd.** *Redis Documentation: Eviction Policies*. Documentación técnica sobre las políticas de retención en memoria (LRU) y manejo de *Time-To-Live* (TTL) en la capa de caché. Recuperado de: <https://redis.io/docs/manual/eviction/>
8. **Docker Inc.** *Docker Compose Documentation*. Guía de referencia para el aislamiento de procesos, mapeo de volúmenes compartidos y escalamiento horizontal de los nodos consumidores. Recuperado de: <https://docs.docker.com/compose/>